Rapport De Réalisation Technique

Projet TOPOS : Gestionnaire des Emplacements de Restaurants

Candidature au poste de Développeur Full-Stack

À l'attention de l'équipe technique de Geoptis

Réalisé par : Melissa MEDJBER

Date de rendu : Avril 2026

Dépôt GitHub : https://github.com/Meli-ileM/restaurant-locator-test.git

Sommaire

1. Introduction 2
2. Objectifs et Défis 2
    - Objectifs du projet 2
    - Défis techniques identifiés 2
3. Analyse et Conception 3
    - Méthodologie de travail 3
    - Diagramme du système 3
    - Plan de développement 4
4. Réalisations 4
    - Choix techniques et Environnement de développement 4
    - Architecture du code 5
    - Charte graphique : 5
    - Solutions aux défis: 7
5. Tests et Validations 7
6. Problèmes rencontrés et Solutions 10
    - Problème 1 : Persistance de la précision décimale entre les couches 10
    - Problème 2 : Conflit visuel entre la carte Leaflet et la couche UI 11
    - Évolution prévue : 11
7. CONCLUSION ET REMERCIEMENTS 11

1. Introduction

Le présent projet, intitulé **TOPOS**, a été conçu et développé dans le cadre du processus de recrutement pour le stage de développeur Full-Stack au sein de l'entreprise **Geoptis**. Cette réalisation ne se limite pas à une simple réponse technique à un cahier des charges ; elle se veut une démonstration de la capacité à concevoir une solution logicielle cohérente, où la donnée géographique est traitée comme un actif critique nécessitant une intégrité absolue.

Le secteur de la géomatique, cœur de métier de Geoptis, impose des standards de précision et de fiabilité qui ont guidé chaque décision architecturale de ce système. **TOPOS** — terme issu du grec ancien signifiant à la fois le lieu, l'emplacement et la structure logique — est une application de gestion d'emplacements de restaurants qui place la validation géospatiale au centre de son moteur.

L'approche retenue privilégie une séparation claire des préoccupations (Separation of Concerns) et une expérience utilisateur raffinée, reflétant ainsi l'exigence d'une plateforme capable de transformer des coordonnées mathématiques brutes en informations métier exploitables et visuellement structurées. Ce document détaille le cheminement intellectuel, les choix technologiques et les méthodologies de validation qui ont permis de donner vie à cette plateforme.

2. Objectifs et Défis

Objectifs du projet

L'objectif central de cet exercice est de construire une application fonctionnelle de gestion d'emplacements de restaurants, simulant les plateformes géospatiales réelles développées par Geoptis. Les buts principaux sont les suivants :

1. **Gestion des emplacements :** Permettre aux utilisateurs d'ajouter et de visualiser des restaurants associés à des coordonnées géographiques précises.
2. **Intégrité des données :** Garantir la précision et la validation rigoureuse des informations de localisation, considérées comme critiques pour ce type d'application.
3. **Expérience utilisateur :** Offrir une interface fluide permettant la recherche et le filtrage dynamique des établissements.
4. **Démonstration méthodologique :** Au-delà du résultat final, l'objectif est d'exposer un processus de réflexion clair, une documentation appropriée et des décisions techniques structurées.

Défis techniques identifiés

Le cahier des charges impose de relever plusieurs défis spécifiques liés à la nature des données manipulées :

- **Validation géographique côté client et serveur :** S'assurer que les coordonnées respectent les plages valides (Latitude entre -90 et 90, Longitude entre -180 et 180) et rejeter toute valeur incohérente.
- **Gestion de la précision décimale :** Garantir que le système supporte et conserve la précision des coordonnées jusqu'à 8 décimales, tant au niveau du transfert API que du stockage en base de données.

- **Filtrage et recherche performants :** Implémenter une logique permettant de retrouver des restaurants par nom, adresse ou type de cuisine de manière réactive.
- **Fiabilité du stockage :** Concevoir un schéma PostgreSQL incluant des contraintes (`CHECK constraints`) pour assurer la sécurité des données directement au niveau de l'infrastructure.
- **Traitement des cas limites :** Gérer les scénarios critiques tels que les coordonnées aux pôles ou sur la ligne de changement de date.

3. Analyse et Conception

Méthodologie de travail

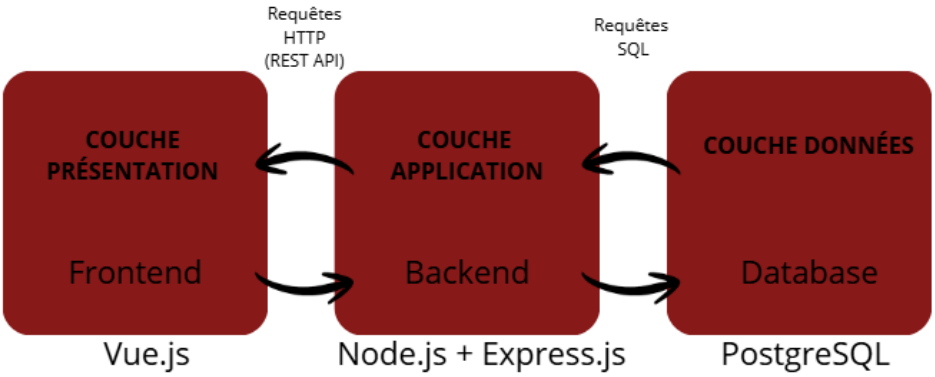
Pour garantir la qualité et la livraison des fonctionnalités dans le temps imparti, J'ai adopté une méthodologie rigoureuse basée sur les principes suivants :

- Approche Agile & Découpage des tâches : Le projet a été segmenté en micro-objectifs itératifs. J'ai priorisé le développement du "noyau" (Backend et Base de données) avant d'entamer la couche utilisateur. Ce découpage a permis de tester chaque point de terminaison API indépendamment du Frontend.
- Contrôle de version (Git) : L'utilisation de Git a permis une gestion saine des évolutions du code, avec des messages de commit explicites pour tracer l'avancement de chaque fonctionnalité (Backend, Frontend, Design).
- Conception "API-First" : En définissant précisément les contrats d'interface (points de terminaison, format des requêtes JSON) dès le début, j'ai assuré une intégration fluide entre le serveur Node.js et l'interface Vue.js.

Diagramme du système

L'architecture de l'application repose sur un modèle Client-Serveur découplé, garantissant une séparation nette entre la logique de présentation et la logique de persistance des données.

- Couche Présentation (Frontend) : Développée avec Vue.js 3, elle gère l'état de l'interface, les validations de formulaires et l'affichage dynamique des données filtrées.
- Couche Application (Backend) : Le serveur Express agit comme un orchestrateur. Il reçoit les requêtes, applique les validations métier (plages de coordonnées) et communique avec la base de données.
- Couche Données (Database) : PostgreSQL assure la persistance. C'est ici que sont appliquées les contraintes d'intégrité finales via des règles SQL strictes (`CHECK constraints`).



Plan de développement

1. Modélisation : Création de la table restaurants avec les types de données géographiques appropriés.
2. Infrastructure Backend : Mise en place des routes API et des validateurs de coordonnées.
3. Interface Frontend : Développement du Dashboard, de la sidebar de navigation et du système de formulaire modal.
4. Intégration & Tests : Liaison des composants aux endpoints API et validation finale via Thunder Client.

4. Réalisations

Choix techniques et Environnement de développement

Pour la réalisation de ce projet, j'ai respecté les prérequis technologiques de Geoptis tout en privilégiant une approche **"Lightweight"** (légère) et performante. L'objectif était de garder un contrôle total sur le flux de données sans dépendances tierces superflues.

Environnement de travail et outils :

- **IDE :** Visual Studio Code pour le développement.
- **Versionnage :** GitHub pour la gestion du code source.
- **Base de données :** Supabase pour l'hébergement de l'instance PostgreSQL.
- **Tests API :** Thunder Client (intégré à VS Code) pour la validation des points de terminaison.

Stack Technologique (Backend) :

- **Node.js & Express 5 :** Utilisation de la dernière version d'Express pour bénéficier des meilleures performances de routage.
- **Pilote pg (Native Driver) :** J'ai fait le choix de ne pas utiliser d'ORM (comme Prisma ou Sequelize) pour manipuler les données. Cela permet d'écrire du SQL pur, garantissant une exécution optimale des requêtes et une maîtrise totale des contraintes de précision.

- **Cors & Dotenv :** Pour la sécurité des échanges entre domaines et la gestion sécurisée des variables d'environnement (clés de connexion).

Stack Technologique (Frontend) :

- **Vue.js 3 (Composition API & <script setup>) :** Utilisation des standards modernes de Vue pour une réactivité fluide.
- **Vite :** Pour un environnement de build ultra-rapide.
- **Leaflet :** Choisi pour sa légèreté et sa robustesse dans l'affichage des données cartographiques.
- **Fetch API :** J'ai privilégié l'API native du navigateur plutôt qu'une bibliothèque externe comme Axios, afin de réduire la taille du bundle final.
- **CSS Natif :** L'intégralité du design **TOPOS** repose sur du CSS moderne (Variables CSS, Flexbox, Grid) sans framework UI (type Vuetify), permettant une identité visuelle unique et un rendu extrêmement rapide.

Architecture du code

L'application est découpée en deux entités indépendantes (Client et Serveur) pour garantir une séparation nette des responsabilités :

- **Backend (Pattern MVC) :**
  - **Models (restaurantModel.js) :** Gère exclusivement les interactions avec PostgreSQL via des requêtes SQL paramétrées pour éviter les injections.
  - **Controllers (restaurantController.js) :** Orchestre la logique métier. C'est ici que j'ai centralisé les fonctions de validation des coordonnées avant toute opération en base.
  - **Routes (restaurantRoutes.js) :** Définit les points d'entrée de l'API (GET, POST, SEARCH, FILTER), agissant comme une interface claire pour le frontend.
- **Frontend (Composants Modulaires) :**
  - Le projet est organisé par responsabilités : layout/ pour la structure fixe (**Sidebar**, **TopBar**) et ui/ pour les composants métier (**RestaurantTable**, **RestaurantsMap**, **RestaurantModal**).
  - **Dashboard.vue** joue le rôle d'orchestrateur central, détenant l'état des données (le "State") et le distribuant aux composants enfants via des *props*.

Charte graphique :

L'identité visuelle a été pensée pour refléter le sérieux d'un outil d'ingénierie tout en restant élégante :

- **Identité visuelle et concept :** Le nom **TOPOS** souligne la dimension géographique du projet tout en s'ancrant dans une sémantique profonde. Issu du grec ancien *τόπος*, c'est un concept riche qui voyage entre la géographie, la philosophie et la littérature ; il désigne ici l'articulation précise entre le lieu physique et sa structure logique. Cette dualité définit une application où chaque coordonnée devient un point de convergence entre la rigueur mathématique du code et la réalité urbaine. Pour illustrer cette vision, le logo a été intégré, garantissant une netteté parfaite sur tous les écrans, tout en reprenant les codes graphiques de la cartographie classique.

NOM DE L'APP

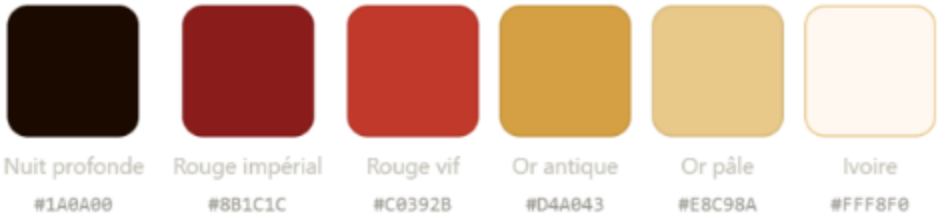
TOPOS du grec ancien τόπος — « lieu, emplacement »

Court, mémorable, ancré dans la cartographie. Topos désigne précisément l'emplacement géographique

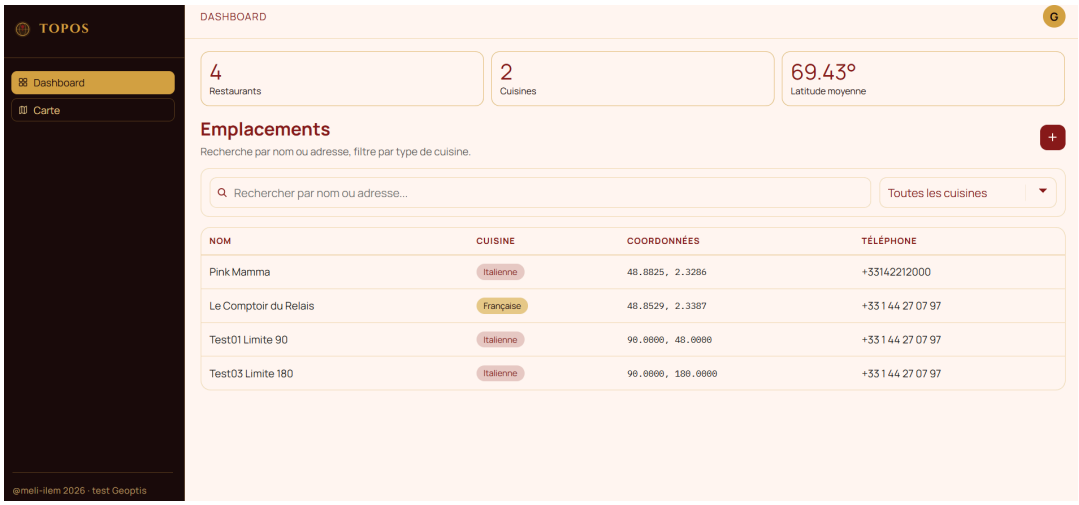
LOGO

- **Palette Chromatique :** J'ai sélectionné le **Rouge Tyrien** pour la solidité des structures et l'**Or Antique** pour les éléments d'interaction (boutons, bordures), créant un contraste "Imperial Greek" professionnel. Ce choix répond également à des impératifs d'ergonomie et de psychologie cognitive : dans le domaine du design d'interface, le rouge est une couleur stimulante associée à l'appétit et à la gastronomie, ce qui renforce la cohérence thématique de l'application. Cette palette, posée sur un fond Ivoire, assure une lisibilité optimale tout en affirmant l'identité visuelle de **TOPOS**.

PALETTE DE COULEURS



- **Interface Dashboard :** L'interface occupe 100% de l'écran pour une immersion totale. J'ai privilégié un **formulaire modal avec effet de flou** (backdrop-filter: blur) pour l'ajout de restaurants, ce qui permet de maintenir l'utilisateur dans son contexte de travail tout en isolant visuellement la tâche de saisie.



Solutions aux défis:

Le code a été conçu pour répondre de manière concrète aux enjeux géospatiaux identifiés en amont, en garantissant la fiabilité de la donnée et la performance de l'interface.

- **Précision décimale :** Pour limiter les erreurs d'arrondi liées aux types flottants classiques (IEEE 754), j'ai configuré le stockage en base de données avec un type **DECIMAL**, tel qu'indiqué dans la couche d'accès aux données. Ce choix permet de conserver un niveau de précision élevé sur les latitudes et longitudes, compatible avec une exploitation géospatiale fine. Lors de l'insertion, les valeurs reçues sont converties explicitement en nombres via `parseFloat()`, ce qui aligne parfaitement la couche applicative avec les contraintes de précision de la base de données.
- **Validation des coordonnées :** Une fonction de validation dédiée contrôle successivement la présence de la donnée, sa nature numérique, puis son appartenance aux bornes géographiques autorisées, soit **[-90, 90]** pour la latitude et **[-180, 180]** pour la longitude. Cette logique en entonnoir évite l'enregistrement de points incohérents ou physiquement impossibles à représenter sur une carte, protégeant ainsi l'intégrité du système.
- **Réactivité du filtrage :** Afin d'éviter une sollicitation excessive du serveur à chaque frappe clavier, j'ai mis en place un mécanisme de **debounce** dans l'interface Vue.js. L'application attend **250 ms** après la dernière saisie de l'utilisateur avant d'émettre la requête HTTP. Cette optimisation améliore à la fois les performances réseau et le confort d'utilisation en offrant une recherche fluide et sans saccades.

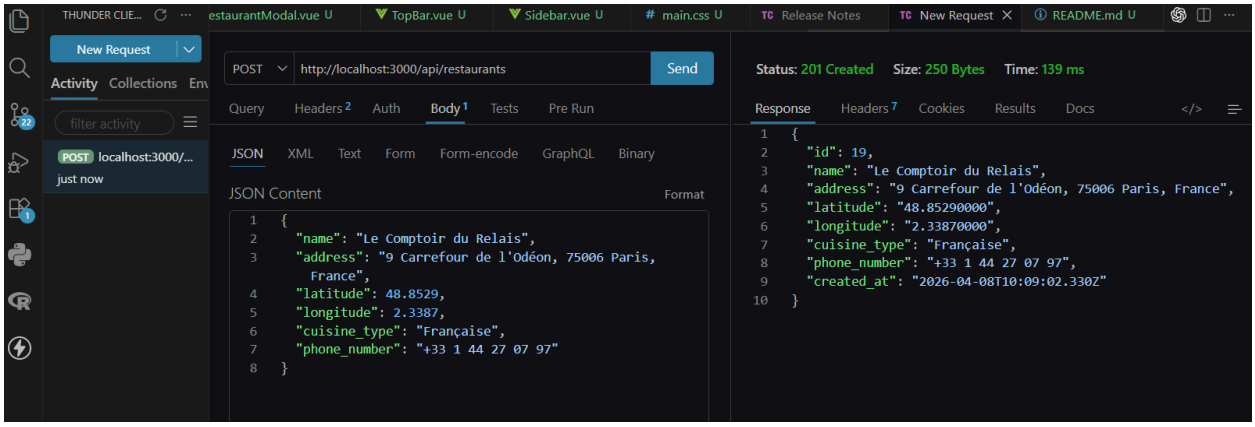
5. Tests et Validations

C'est ici que je démontre la robustesse de ma solution via des preuves concrètes. Les tests ont été réalisés avec **Thunder Client** pour valider le comportement du backend face à des données réelles et des cas limites avant de le relier au frontend.

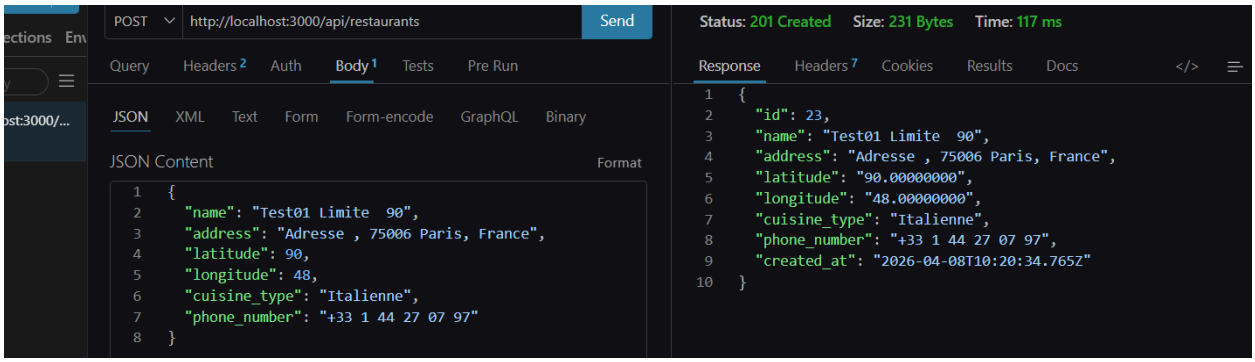
A. Scénario Nominal (Succès) :

- **Test :** Insertion d'un restaurant avec des coordonnées valides et une précision de 8 décimales.

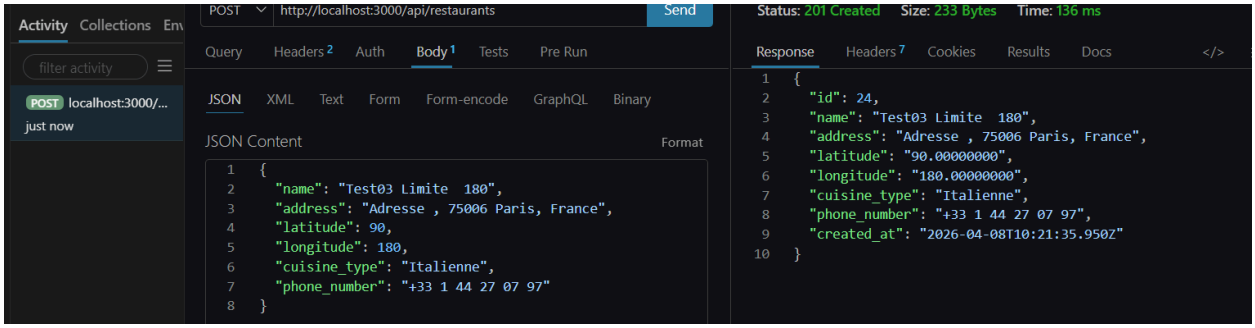
- **Résultat :** Statut 201 Created. La réponse JSON confirme que les données sont enregistrées sans aucun arrondi.



- Test limite latitude de 90

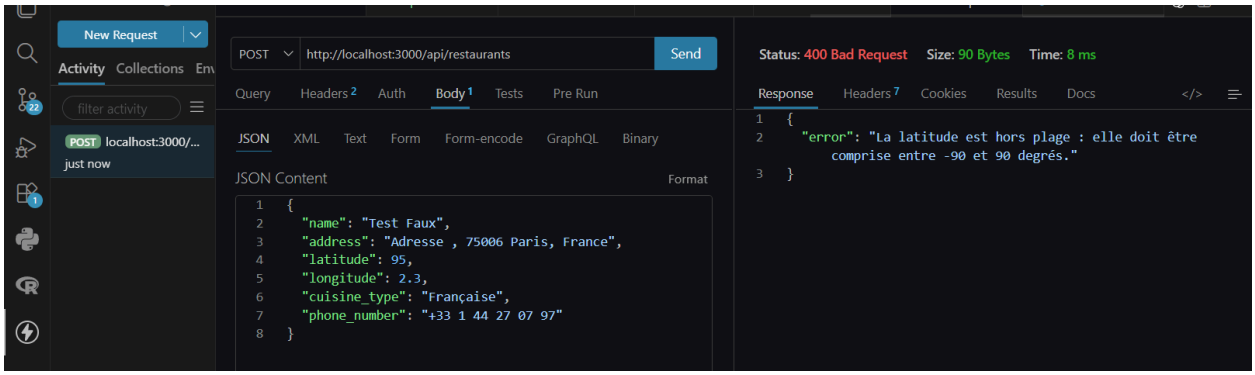


- Test limite longitude de 180

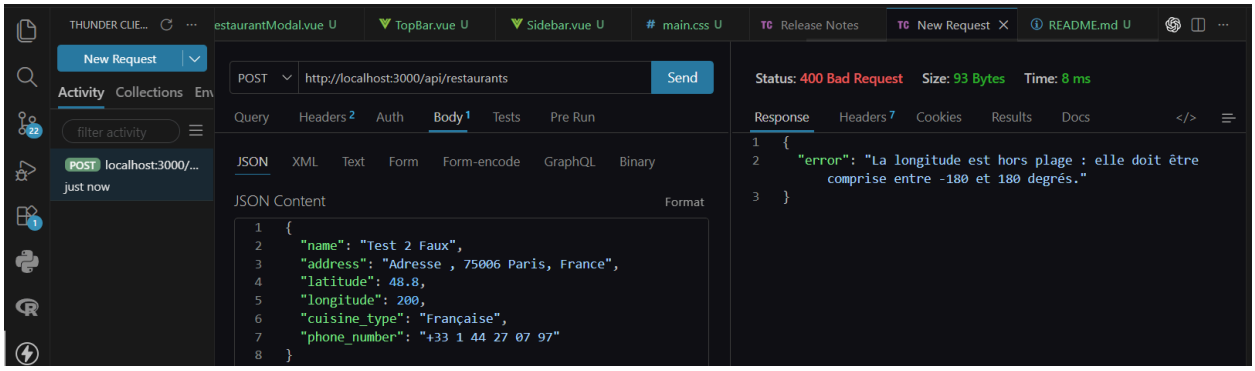


- Scénarios d'erreurs normaux (hors plage de valeurs)
- Un test avec une Latitude hors plage de valeurs:

- **Résultat :** Statut 400 Bad Request. Le serveur renvoie un message explicite : *"La latitude est hors plage : entre -90 et 90 degrés.*

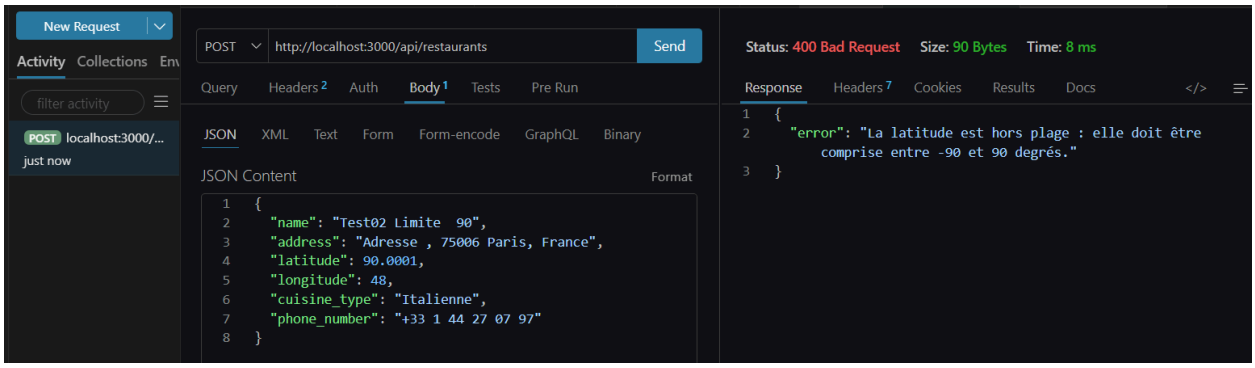


- Un autre test avec une Longitude hors plage de valeurs:
- **Résultat :** Statut 400 Bad Request. Le serveur renvoie un message explicite : *"La longitude est hors plage : entre -180 et 180 degrés.*



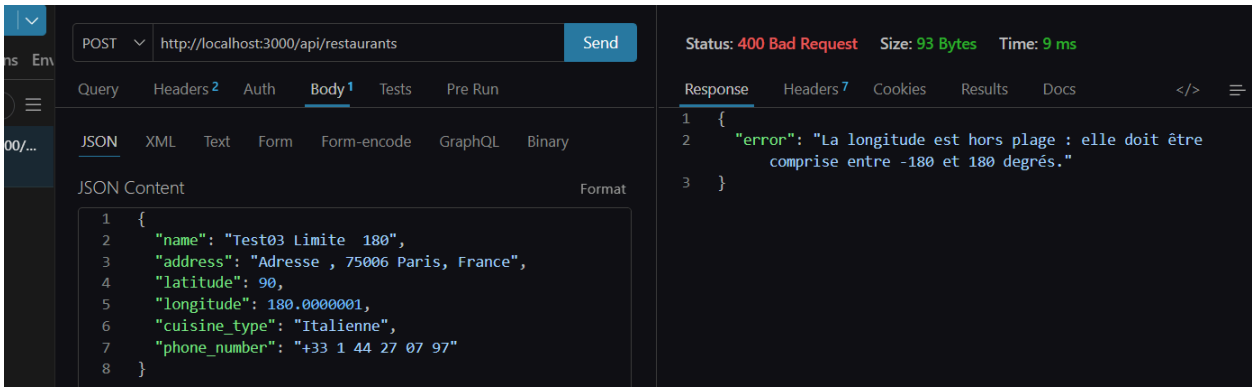
B. Scénario Limite (Géographie) :

- **Test :** Tentative d'insertion d'une latitude de 90.000001 (hors plage).
- **Résultat :** Statut 400 Bad Request. Le serveur renvoie un message explicite : *"La latitude est hors plage : entre -90 et 90 degrés.*



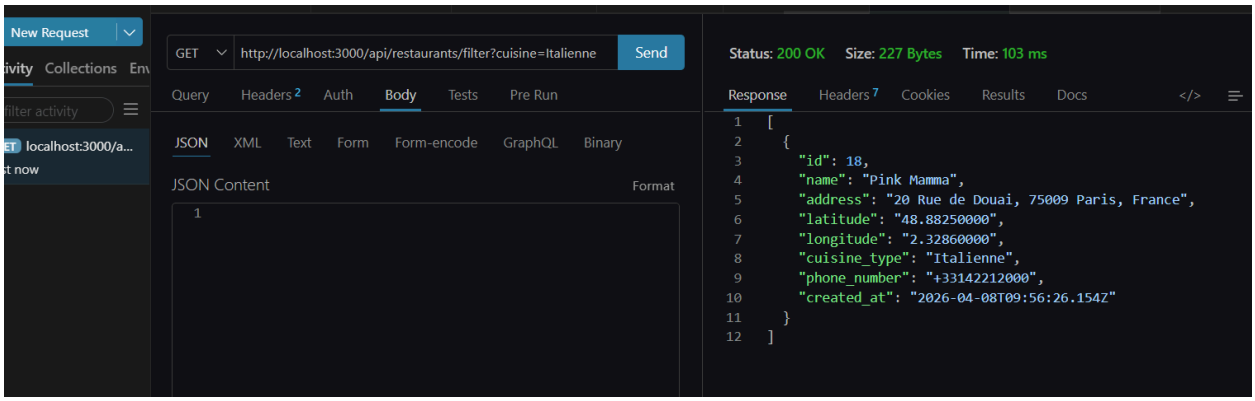
- Un test avec une longitude hors plage (limite)

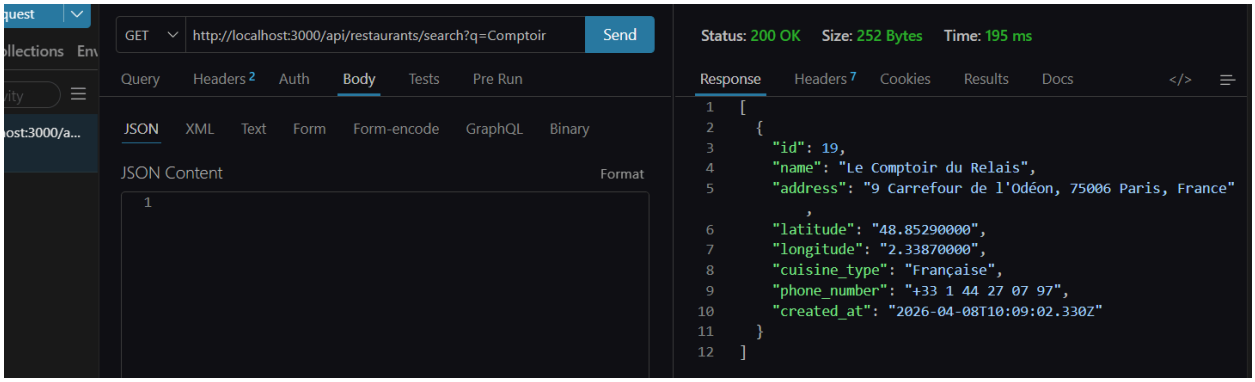
- **Résultat :** Statut 400 Bad Request. Le serveur renvoie un message explicite : *"La longitude est hors plage : entre -180 et 180 degrés.*



C. Scénario de Recherche et Filtrage :

- **Test :** Appel du point de terminaison /api/restaurants/filter?cuisine=Italienne.
- **Résultat :** Statut 200 OK. Seuls les restaurants correspondants sont retournés dans le tableau d'objets.





6. Problèmes rencontrés et Solutions

Au cours du développement de la plateforme **TOPOS**, j'ai dû faire face à des défis techniques qui ont nécessité une analyse ciblée pour garantir la fiabilité du système.

Problème 1 : Persistance de la précision décimale entre les couches

- **Le défi :** Lors des premiers tests d'intégration, certaines coordonnées envoyées par le frontend présentaient des écarts d'arrondi potentiels liés à la représentation flottante native de JavaScript (**IEEE**

754). Or, l'exigence métier de Geoptis imposait la conservation d'une précision élevée sur les coordonnées géographiques pour garantir la fiabilité des données de localisation.

- **La solution :** J'ai appliqué une stratégie en deux niveaux. D'abord, j'ai retenu un stockage en `DECIMAL` côté PostgreSQL pour garantir la précision persistée. Ensuite, côté backend, j'ai ajouté une conversion explicite (`parseFloat`) et une validation stricte des entrées (présence, type numérique, bornes géographiques). Cette combinaison sécurise la qualité de la donnée tout au long du flux, de la saisie utilisateur jusqu'à la persistance en base.

Problème 2 : Conflit visuel entre la carte Leaflet et la couche UI

- **Le défi :** Pendant l'intégration cartographique, la modale de création de restaurant pouvait être perturbée par le contexte de rendu de la carte (**stacking context / z-index**), ce qui nuisait à la superposition et à l'effet visuel attendu.
- **La solution :** J'ai utilisé le composant `Teleport` de Vue 3 pour conserver la logique du composant dans le Dashboard tout en rendant le DOM de la modale directement sous l'élément `body`. Cette séparation a supprimé les conflits de superposition avec Leaflet et a permis d'appliquer correctement la propriété `backdrop-filter: blur()`, garantissant une interface fluide et sans artefacts visuels.

Évolution prévue :

- **Ajout des indicateurs géographiques**

À ce stade, l'application calcule et affiche uniquement la latitude moyenne.
Cet indicateur permet déjà une lecture synthétique de la répartition des restaurants sur l'axe Nord/Sud.

Dans une prochaine itération, il est prévu d'ajouter la longitude moyenne afin de compléter l'analyse sur l'axe Est/Ouest.
L'objectif est d'obtenir une vision bidimensionnelle plus fidèle de la distribution spatiale des établissements, tout en conservant une lecture KPI simple dans le dashboard.

7. CONCLUSION ET REMERCIEMENTS

La réalisation de ce test technique a été l'opportunité de concevoir une application Full-Stack cohérente, en mettant l'accent sur la qualité de l'architecture et l'intégrité des données géospatiales. En respectant les contraintes de temps et les exigences technologiques (Vue 3, Express, PostgreSQL), j'ai pu livrer un outil à la fois performant et robuste.

Le projet **TOPOS** démontre ma capacité à :

1. **Maîtriser une stack technique complète** sans dépendances superflues.
2. **Anticiper les problématiques de précision** critique liées au domaine de Geoptis.
3. **Concevoir une interface utilisateur moderne** et ergonomique.

Je tiens à remercier l'équipe technique de Geoptis pour cet exercice stimulant, qui reflète parfaitement les défis réels du développement de solutions géospatiales.